

Análisis organizado de TallerRNA.ipynb

Archivo: Laboratorios/Taller 3/TallerRNA.ipynb

1. Visión general del cuaderno

1.1 Propósito general

- El cuaderno implementa un experimento completo de clasificación facial con una red neuronal densa tipo MLP sobre el dataset Olivetti Faces.
- El objetivo es entrenar varias variantes del modelo, compararlas con validación y conservar las mejores para evaluarlas después en prueba.
- El flujo general del trabajo es: carga de datos, partición estratificada, normalización, construcción de modelos, búsqueda aleatoria de hiperparámetros, selección top-3 y análisis final.

1.2 Dataset utilizado

- Se trabaja con el conjunto Olivetti Faces, formado por 400 imágenes distribuidas en 40 clases, una clase por persona.
- Cada imagen se representa como un vector aplanado de 4096 características, equivalente a una imagen original de 64 x 64 píxeles.

2. Preparación de los datos

2.1 Carga y partición estratificada

- El dataset se carga con `fetch_olivetti_faces` y luego se divide con `train_test_split` en dos etapas.
- Primero se separa 60% para entrenamiento y 40% para un conjunto temporal; después este conjunto temporal se divide en dos partes iguales para validación y prueba.
- La estratificación asegura aproximadamente 6 imágenes por identidad en entrenamiento, 2 en validación y 2 en prueba.

2.2 Verificación visual de la estratificación

- El cuaderno incluye una función que muestra las 10 imágenes de una persona concreta repartidas entre entrenamiento, validación y prueba.
- Esto permite comprobar de forma visual que la partición se hizo correctamente y que cada identidad quedó representada en los tres subconjuntos.

2.3 Analogía para entender la división de datos

- El entrenamiento puede entenderse como el libro de texto y los ejercicios con los que el modelo estudia.
- La validación funciona como un examen de práctica usado para revisar el progreso y ajustar la estrategia, pero sin aprender directamente de esos datos.
- La prueba actúa como el examen final, reservado únicamente para el cierre del experimento y para obtener una medición imparcial del desempeño.

3. Normalización y coherencia del flujo

3.1 Qué es la normalización

- La normalización transforma los datos para que las variables tengan una escala comparable y ninguna domine a las demás solo por magnitud numérica.
- En este proyecto, ese proceso se aplica sobre vectores de 4096 características por imagen.

3.2 Cómo se aplica StandardScaler

- El StandardScaler se ajusta exclusivamente con los datos de entrenamiento.
- Una vez calculadas las estadísticas del entrenamiento, esas mismas se aplican para transformar entrenamiento, validación y prueba.
- Este flujo de ajuste y aplicación evita errores conceptuales y garantiza un proceso profesional de preprocesamiento.

3.3 Por qué la normalización es importante

- La normalización estabiliza el entrenamiento en espacios de alta dimensión y favorece una convergencia más rápida y consistente.
- También evita la fuga de información, porque impide que validación o prueba influyan en el cálculo del preprocesamiento.

3.4 Uso correcto en validación y prueba

- Los conjuntos de validación y prueba deben estar normalizados antes de entrar al modelo.
- Si se usaran sin normalizar, el modelo recibiría datos en una escala diferente a la aprendida durante el entrenamiento y produciría predicciones incoherentes.
- Por eso el accuracy, el classification report y la matriz de confusión se calculan sobre datos ya transformados.

3.5 Cómo se calculan media y desviación estándar

- La media y la desviación estándar no se calculan por clase, sino globalmente sobre todo el conjunto de entrenamiento.
- StandardScaler opera columna por columna, es decir, característica por característica o píxel por píxel.
- Normalizar por clase sería inadecuado porque eliminaría diferencias útiles entre identidades y rompería la referencia común necesaria para clasificar.

4. Reproducibilidad del experimento

4.1 Semillas aleatorias

- El cuaderno fija semillas para NumPy, TensorFlow y random.
- Esto mejora la reproducibilidad y permite que los resultados sean más consistentes entre ejecuciones, aunque el determinismo total también depende de la versión de TensorFlow y del hardware.

5. Diseño de la red neuronal

5.1 Constructor build_model

- La función build_model recibe un perfil arquitectónico, una tasa de aprendizaje y varios hiperparámetros opcionales.
- La entrada del modelo es un vector plano de 4096 características y la salida es una capa softmax de 40 neuronas.

5.2 Perfil Robusto

- Utiliza capas Dense con activación ReLU, inicialización HeNormal, Dropout y optimizador Adam.
- Está orientado a una convergencia estable y a reducir sobreajuste mediante regularización por abandono de neuronas.

5.3 Perfil Profundo

- Emplea capas Dense con activación SELU, inicialización LecunNormal y regularización L2.
- Usa AdamW cuando está disponible y recurre a Adam como alternativa en versiones antiguas de TensorFlow.

5.4 Perfil Adaptativo

- Utiliza capas Dense con HeNormal seguidas de LeakyReLU y optimizador RMSprop.
- Además, emplea ReduceLROnPlateau para reducir la tasa de aprendizaje cuando la validación deja de mejorar.

5.5 Observación técnica sobre la implementación

- El argumento `neurons_list=[]` en la firma de `build_model` usa una lista mutable por defecto, lo cual no es recomendable.
- Una implementación más segura usaría `None` y asignaría la lista dentro de la función.

6. Búsqueda de hiperparámetros

6.1 Esquema general

- El cuaderno realiza una búsqueda aleatoria de 50 iteraciones.
- En cada iteración se selecciona al azar un perfil de red, una tasa de aprendizaje y otros hiperparámetros específicos de esa arquitectura.

6.2 Entrenamiento y callbacks

- Cada modelo se entrena hasta 150 épocas con `batch_size` de 32.
- Se utiliza `EarlyStopping` sobre `val_loss` con restauración de los mejores pesos, y para el perfil Adaptativo se añade `ReduceLROnPlateau`.

6.3 Qué se guarda en cada iteración

- El experimento guarda la mejor precisión de validación, la mejor pérdida de validación, el historial de entrenamiento, los pesos y el tiempo consumido.
- Esto permite reconstruir después los modelos más destacados y revisar su comportamiento con detalle.

6.4 Riesgos operativos

- Guardar historias completas y pesos de 50 modelos puede consumir memoria considerable en equipos limitados.
- Además, ciertas versiones antiguas de TensorFlow pueden usar nombres distintos para algunas métricas del historial.

7. Selección y evaluación de modelos

7.1 Selección del top 3

- Los resultados se ordenan por val_accuracy y se eligen los tres mejores modelos.
- La decisión se toma con validación y no con prueba, lo cual mantiene la evaluación final libre de sesgo.

7.2 Importancia de la validación

- La validación permite prevenir el sobreajuste, comparar modelos de forma justa y ajustar hiperparámetros sin contaminar la prueba.
- Sin este conjunto intermedio, no sería posible saber si el modelo está aprendiendo patrones generales o simplemente memorizando ejemplos de entrenamiento.

7.3 Evaluación en prueba

- Cada modelo del top 3 se reconstruye con sus hiperparámetros originales y con los pesos guardados.
- Después se calcula el test accuracy, se genera el classification report y se construye la matriz de confusión.

7.4 Interpretación del ranking

- El orden del top 3 está definido por validación, por lo que no siempre coincide con el mayor accuracy en prueba.
- Esto es correcto desde el punto de vista metodológico y evita elegir modelos usando el conjunto final de evaluación.

8. Visualización y análisis cualitativo

8.1 Curvas de entrenamiento

- Para cada modelo del top 3 se grafican curvas de loss y accuracy de entrenamiento frente a validación.
- Estas gráficas ayudan a detectar convergencia, estabilidad y posibles señales de sobreajuste.

8.2 Predicciones sobre imágenes de prueba

- Una función selecciona imágenes aleatorias del conjunto de prueba y muestra la predicción junto con la clase real.
- Esta inspección permite revisar aciertos y errores de manera visual e intuitiva.

8.3 Visualización de errores

- Otra función identifica los errores de clasificación y los presenta en una galería de imágenes.
- Esto facilita analizar qué identidades se confunden y si esos fallos resultan razonables visualmente.

9. Conclusiones del cuaderno

9.1 Hallazgos principales

- El perfil Robusto parece ser el que mejor generaliza en este dataset pequeño.
- Las curvas muestran una convergencia rápida y estable, con poca separación entre entrenamiento y validación.
- Esto sugiere que Dropout contribuye de forma importante al control del sobreajuste.

9.2 Rendimiento final

- El cuaderno resume un desempeño cercano al 95% de accuracy en prueba y un macro F1 alrededor de 0.97.
- Los errores se concentran en pocas clases, lo que indica un comportamiento sólido para la mayor parte de las identidades.

Cuadro de síntesis

Bloque	Contenido	Importancia
Preparación	Partición estratificada y normalización.	Define un preprocesamiento correcto y sin fuga de información.
Modelado	Tres perfiles MLP con distintas estrategias.	Permite comparar enfoques arquitectónicos.
Búsqueda	50 iteraciones aleatorias con validación.	Optimiza hiperparámetros sin usar prueba.
Evaluación	Top 3, accuracy, reporte y matriz de confusión.	Mide desempeño real del modelo.
Análisis visual	Curvas, predicciones y errores.	Ayuda a interpretar resultados más allá de una métrica.